

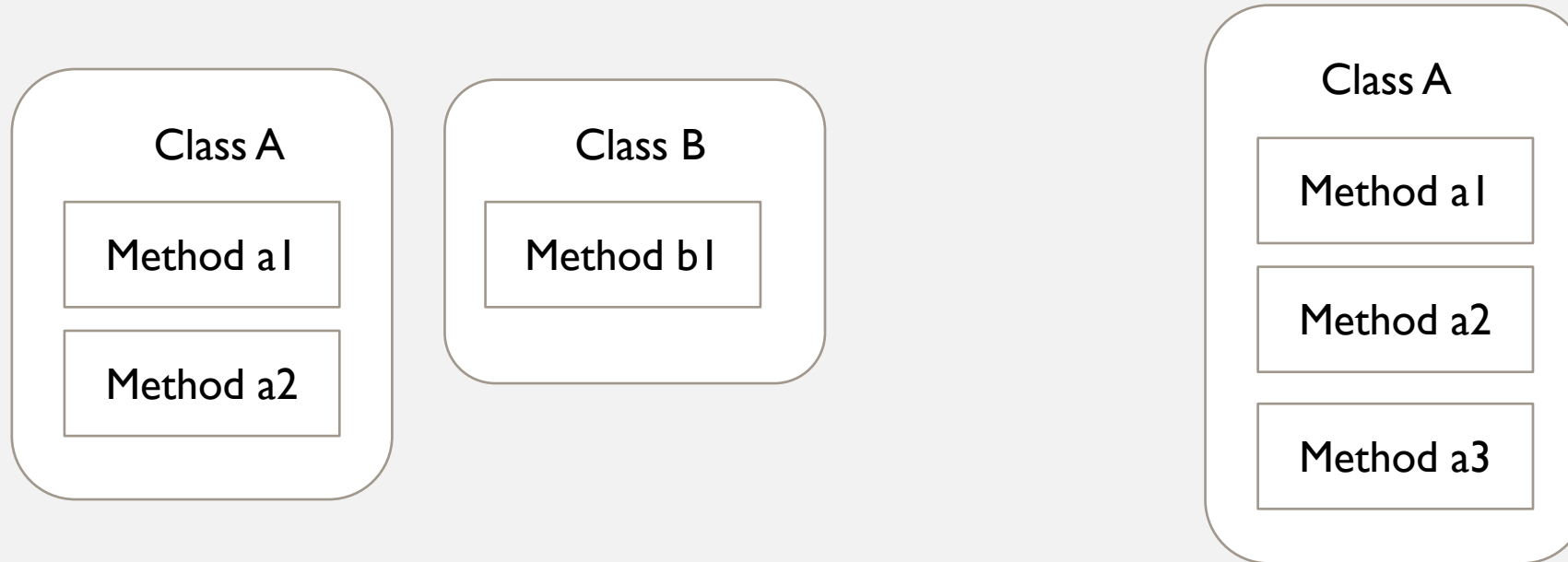
JAVA

Lecture VII – Packages

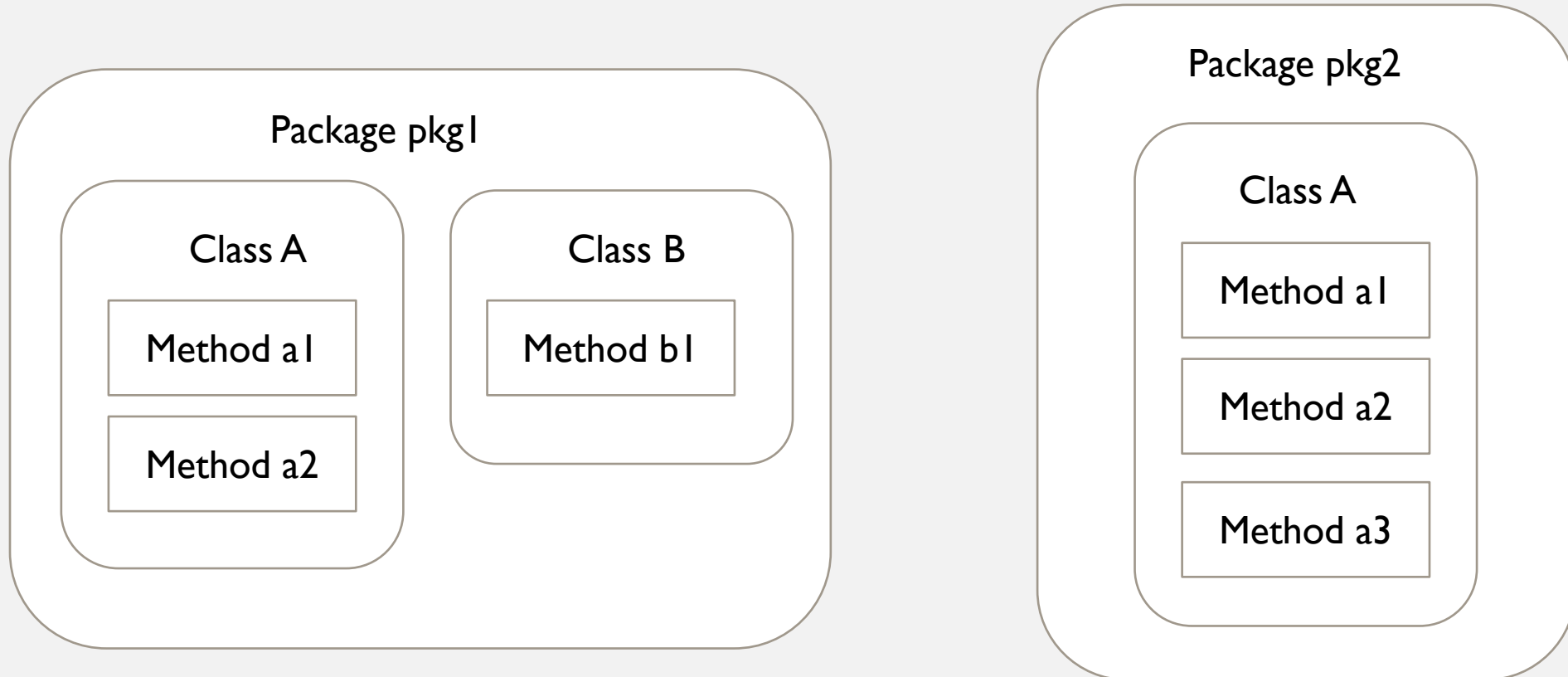
PACKAGE FUNDAMENTALS

- We have classes with different functionalities and different purposes ...
- Packages: a group of related classes and interfaces.
 - Help you organise your code, name a collection of classes
 - Provide another layer of encapsulation, e.g., more access control
- We cannot have two classes of the same name...
- As the programs grow, more and more difficult to find a unique name...
- Solution: package as a namespace.

PACKAGE FUNDAMENTALS



PACKAGE FUNDAMENTALS



DEFINING A PACKAGE

- General form of the package statement:

```
package pkg;           // at the top of a Java file
```

- pkg: package name. By convention, lower case letters are used.
- If no package has been explicitly specified, the default package is used (no name).
- Java uses file system to manage packages, i.e., each package is stored in its own directory.
- Multiple files can be included in the same package.
- A hierarchy of packages:

```
package pack1.pack2.pack3....packN;
```

- Stored in .../pack1/pack2/pack3/.../packN

EXAMPLE

```
package backpack;
class Book{
    private String title, author;
    private int pubDate;
    Book(String t, String a, int d){
        title = t;
        author = a;
        pubDate = d;
    }
    void show(){
        System.out.println("title: " + title);
        System.out.println("author: " + author);
        System.out.println("pubDate: " + pubDate);
    }
}
```

EXAMPLE

```
package backpack;  
class BookDemo{  
    public static void main(String[] args){  
        Book book = new Book("CS", "Me", "2022");  
        book.show();  
    }  
}
```

EXAMPLE

```
package backpack;  
class BookDemo{  
    public static void main(String[] args){  
        Book book = new Book("CS", "Me", "2022");  
        book.show();  
    }  
}
```

Compile the file:

```
javac backpack/BookDemo.java //make sure you are in the  
                               directory above backpack
```

Execute the file

```
java backpack.BookDemo // cannot use java BookDemo
```


COMPILE AND EXECUTE

```
javac backpack/BookDemo.java
```

It compiles the file `BookDemo.java` in the directory `backpack`.

`BookDemo.class` will be generated in the same directory.

`Book.class` will be automatically generated as before.

```
javac -d . BookDemo.java Book.java
```

It creates a new directory (with package name) and stores all class file in the new directory

Execute the file

```
java backpack.BookDemo           // cannot use java BookDemo
```

IMPORTING PACKAGES

- In previous examples, we need to fully qualify the name of the class in different packages...

```
bookpack.Book
```

- **import** statement: bring together classes in different packages.

```
import pkg.classname;
```

- Import entire package:

```
import pkg.*;
```

- No longer need to qualify the whole package name, i.e., use Book rather than bookpack.Book
- The import statement goes after package statement and before class definition.

IMPORTING PACKAGES

- **import** statement can also be used to import Java standard packages. (Java API)
- Most of them start with **java**.

Sub-package	Description
java.lang	Contains a large number of general-purpose classes
java.io	Contains the I/O classes
java.net	Contains those classes that support networking
java.applet	Contains classes for creating applets
java.awt	Contains classes that support the Abstract Window Toolkit
java.util	Contains various utility classes and the Collections Framework

- The java.lang package is automatically imported.

STATIC IMPORT

- **import** followed by **static** allows you to import static member of a class.
- Instead of calling through their class name, we could call them directly.

```
import static java.lang.System.*;

class A{
    public static void main(String[] args){
        out.println("Static import");
    }
}
```